

Report: Hierarchical Memory for Long-Horizon LLMs

Core idea

Your idea can be framed as:

Move old context from expensive full KV state into cheaper compressed / symbolic / summary / graph memory, then reconstruct or retrieve details only when needed.

This sits between four fields:

1. KV cache compression
2. Long-context transformer memory
3. Agent memory systems
4. Graph / hierarchical retrieval memory

1. Long-context transformer memory

These works modify the model architecture so it can remember beyond the normal context window.

Transformer-XL — Reuses hidden states from previous segments using recurrence. Early version of “keep past neural state around.”

Compressive Transformer — Keeps recent memory at high fidelity, compresses older memory.

Very close to your “raw KV → compressed memory” hierarchy.

Memorizing Transformers — Stores past key-value-like representations in external kNN memory.

Closest to “retrieve neural state later.”

Takeaway: These works prove that hierarchical neural memory can work, but they do not fully solve agentic, million-turn memory management.

2. Retrieval-augmented LLMs

These systems keep knowledge outside the context window and retrieve it when needed.

REALM — Trains a language model with a latent document retriever. Shows modular external memory can beat purely parametric memory.

RETRO — Retrieves chunks from a massive 2-trillion-token database and cross-attends to them.

Shows retrieval can substitute for model scale and extend memory.

Classic RAG — Retrieve relevant text chunks into the prompt. Production baseline, but mostly flat retrieval, not tree traversal.

Takeaway: RAG retrieves text, but your idea is richer: retrieve through a structured hierarchy, not just similarity search.

3. Agent memory systems

These are closest to practical “LLM memory operating systems.”

MemGPT — Treats LLM context like OS virtual memory, moving information between memory tiers. Very relevant: explicit memory hierarchy and context paging.

Letta — Production evolution of MemGPT with core memory, archival memory, recursive summaries, and recall search. Directly implements tiered agent memory.

Generative Agents — Stores observations, synthesizes reflections, retrieves memories for planning. Strong example of raw events → higher-level reflections.

MEMORYLLM — Adds a latent memory pool that can self-update over time. Moves toward model-internal updatable memory.

Takeaway: Agent-memory systems already use hierarchy, but mostly over text memories, not full KV states.

4. Graph / knowledge-tree memory

This is where your “Geography → Europe → France” tree idea fits best.

GraphRAG — Builds LLM-derived knowledge graphs, detects communities, summarizes them, and retrieves through graph structure. Very close to concept-tree memory.

Graph-based agent memory — Stores entities, relationships, events, and summaries. Closest to “semantic state instead of token state.”

Knowledge-augmented generation variants — Use structured knowledge graphs for retrieval and reasoning. Useful for traversing concepts instead of retrieving flat chunks.

Takeaway: GraphRAG is probably the strongest public direction for “memory as structured understanding,” but it does not preserve or reconstruct KV.

5. KV cache compression and eviction

This is the hardware-adjacent line of work.

ChunkKV — Compresses KV by preserving semantically informative chunks instead of individual tokens. Very aligned with your “unique information” framing.

ShotKV — Handles prefill and decoding differently while preserving shot-level semantic coherence. Important because reasoning tasks degrade under naive compression.

RocketKV — Two-stage KV compression: coarse eviction plus fine-grained sparse attention.

Hardware-relevant: targets memory bandwidth and capacity savings.

HCAAttention — Combines quantization, value offload, and dynamic KV eviction under GPU/CPU memory constraints. Very relevant to heterogeneous hardware memory hierarchy.

Takeaway: KV compression is advanced, but most methods still decide importance from attention/statistics, not from a semantic hierarchy.

The gap your idea targets

Current systems usually do one of these:

KV compression: keep/drop neural states

RAG: retrieve text chunks

Agent memory: summarize conversation

GraphRAG: build concept graph

Your idea combines them:

Raw KV

↓

Compressed KV

↓

Semantic chunk summary

↓

Concept node

↓

Knowledge tree / graph

↓

Selective reconstruction

That combination is not fully solved.

Best research framing

A strong title for this direction could be:

Hierarchical Semantic KV Memory for Long-Horizon Agentic LLM Inference

The central research question:

Can an LLM demote old context from full KV into compressed semantic memory, preserve pointers to raw text/KV, and later reconstruct only the pieces needed for reasoning?

Why this is important for hardware

For agentic workloads, memory cost grows with total history:

Cost $\approx$ all prior tokens $\times$ KV size

Your direction tries to make it grow with:

Cost $\approx$ active reasoning state + compressed semantic memory

That is the real prize. It turns LLM memory from a flat token buffer into something more like:

L1/L2 cache $\rightarrow$ HBM KV $\rightarrow$ CPU KV $\rightarrow$ SSD KV $\rightarrow$ summaries $\rightarrow$ graph memory

The most promising opportunity is not just “compress KV more.” It is:

Build hardware/runtime support for moving information between neural KV, compressed KV, text memory, and concept-graph memory.